

Extend MPLAB[®]X features with Python

(module4: extending MPLABX with Python)

Stefan Luethin
December 2023

Agenda

1: what is the *MPLABX* ecosystem *MDBCore*, *SDK* -> how to use for automated FW testing

2: compile without IDE

3: Automated *FW-testing* with different *MDBCore* use-models

- a. simple *mdb* -> DOS/bash-script, easy to use but has limits
- b. more advanced *mdbc*s -> *.jar module needs scripting-language with Java-interface, more complex but solves *mdb* limits and opens door into Java-universe
- c. full, but most complex usage of *MDBCore* with Java -> SDK

4: Extending *MPLABX* functionality with Python

5: summary

M6: Short review of module1+2+3a+3b+3c

- module1/intro: introduction into topic
 - Why adding automation to FW-development and MPLABX structure (where does `MDBC` fit in?)
 - Terminology: three `MDBC`-IFs for **automated FW-testing**: `mdb` -> `mbcs` -> `MDBC` (only 3 supported IFs!)
 - `mdb`: `mdb.bat` = `java -jar mdb.jar` -> controlled with `commandFile` -> easy IF with limits
 - `mbcs`: `mbcs.jar` requires scripting-language with Java-IF -> solves `mbbat`-limits ...
 - `MDBC`: `java` modules that provide full functionality though highest complexity
 - Python-IF to **extend MPLABX-functionality** -> example `autoload.py` (**=>this module mod5**)
 - MPLAB^(R)X SDK: docu, NB-projects for `mdb.jar` & `mbcs.jar` / `MDBC.java` examples
- module2/compile: steps to compile your FW into a debug'able *.elf?
 - Get FWrevision from SCM (=git checkout revA -> configuration.xml) -> Update Makefiles `prjMakefilesGenerator.bat`
-> Use `make` to compile sources into *.elf using `nbproject\Makefile*` (-> OUT: *.elf)
- module3a/mdb:
 - simple `MDBC`-interface , `commandLine` interface easy but limits , no extra installation but comes with every MPLABX-version
- module3b/mbcs:
 - wraps major `MDBC`-functionality to ease usage, way simpler then `MDBC`, needs scripting language with Java-IF
- module3c/mdbcore:
 - Full `MDBC`-functionality though need extensive Java-knowledge

M6: autoload.py and the python-IF to MPLABX

Why a Python-IF in MPLAB^(R)X ?

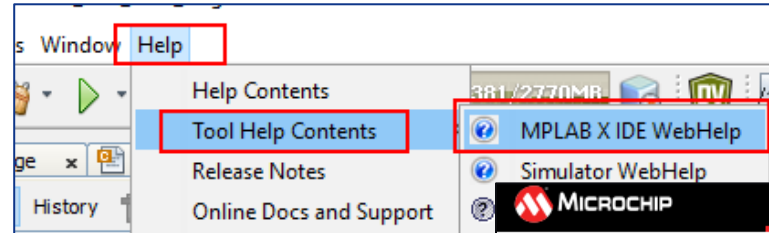
- v5.20 of MPLAB^(R)X, mostly Java, introduced a Python interface
- With this Python-IF you can access MDBCore-classes or its Java-API from a Python-function
- one reason was to support our MPUs but why?
 - MPUs don't have a flash to store the image-to-run, but run out of DDR, which is not visible after reset
 - After reset the MPU only sees the internalSRAM but the DDR space is not yet visible
 - The DDR is set up by a bootloader, called at91, which first has to be loaded from somewhere into internalSRAM and executed - only then the application can be loaded into DDR and executed
 - Developing an application in MPLABX means the IDE has to *download* this at91-bootloader into internalSRAM, *execute* it and then *load the application into DDR*
 - > this is done by a python-script stored in the DFP as one complex/advanced example what you can do with the MPLABX-python-IF
- But you can also use this Python-IF for other purposes as shown in this module
- Since MPLABX-v5.20 every release brings an example called [autoload.py](#) in <mplabx_install_dir>\docs\

M6: documentation of python-IF

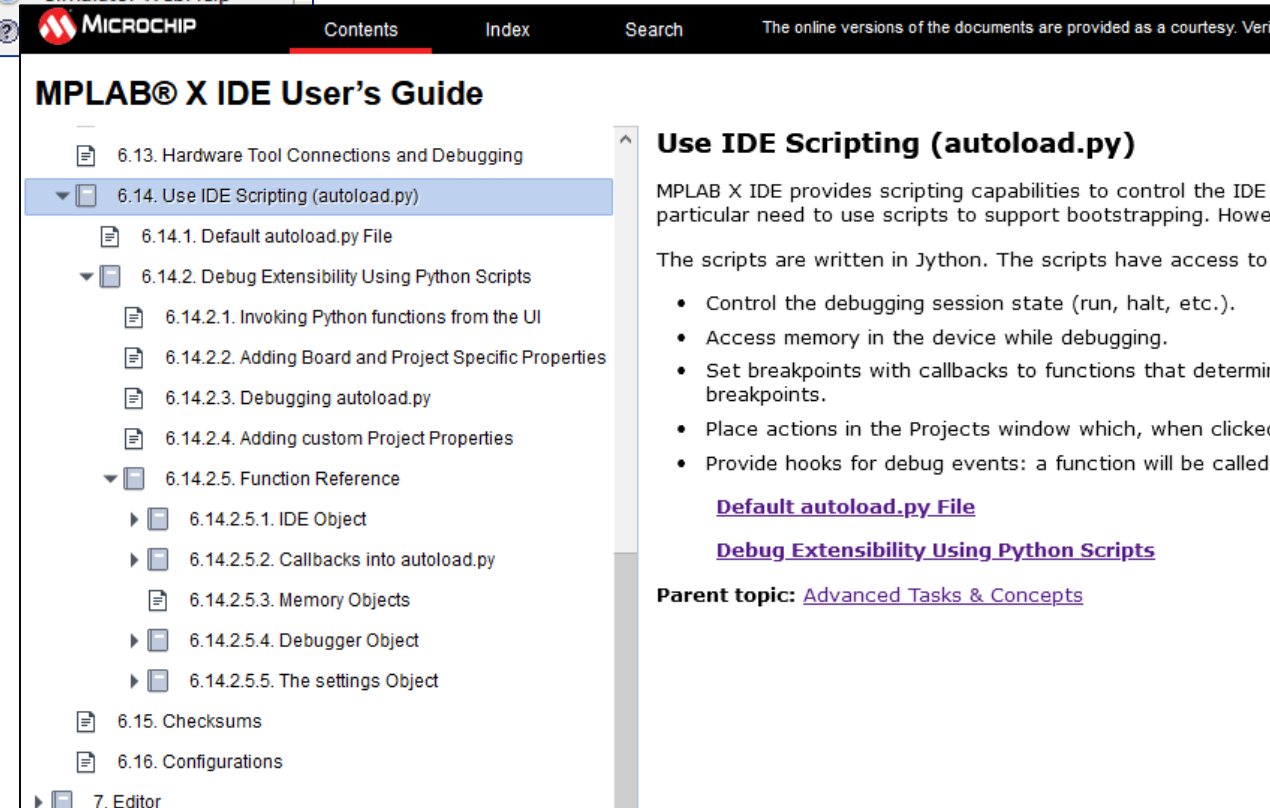
How can you use the Python-IF and autoload.py -> documentation

- To use the Python-IF, just place a file called `autoload.py` into your project.X directory and it will be automatically picked (next slide)

- documentation is available inside MPLABX



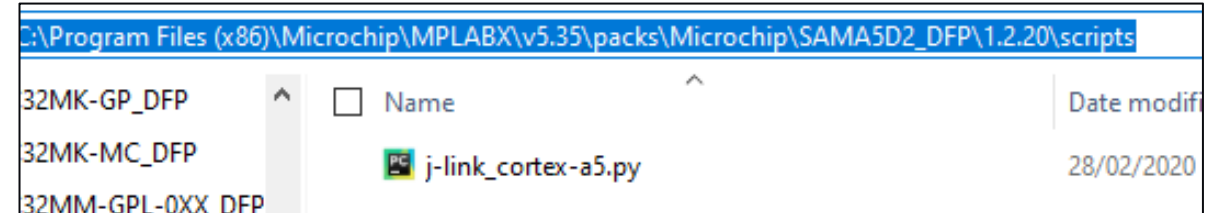
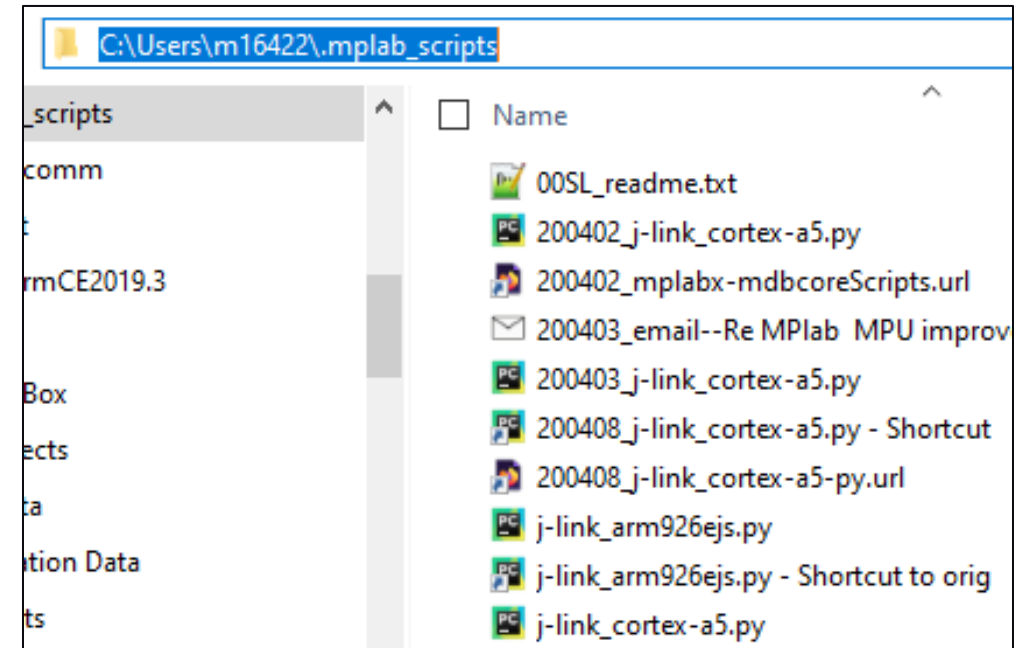
- the available MDBCore-API you can use, as mentioned, is described in the SDK, as explained in mod5/MDBCore



M6: python-scripting in MPLABX and autoload.py

To use python-scripting in MPLABX:

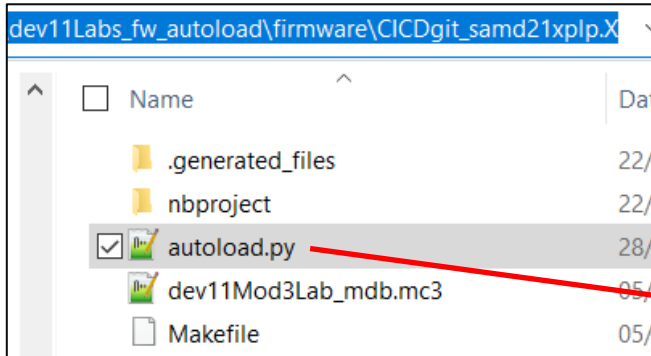
- Just place a file called `autoload.py` into any of the 3 directories MPLABX searches, which are (MPLABX searches in this order and stops if it finds one!):
 1. `C:\users\<id>\.mplab_scripts\` (hidden!)
 2. Local project-dir \$prj.X
 3. path-to-used-DFP (eg: `<X-installD>\packs\Microchip\SAMA5D2_DFP\1.2.20\scripts\`)
- script-search-order can be useful to override or fix something in the MPLABX/DFP
- Remember that the DFPs contains scripts that control debugging -> for MPUs the `<DFP>\scripts\<debug>_<arch>.py` are use to download+execute the at91-bootloader as explained before
- every MPLABX-release since v5.20 brings an example `autoload.py` in `<mplabx>\docs\` -> this is your starting point – just copy into your prj.X and it'll be picked (->next slide)



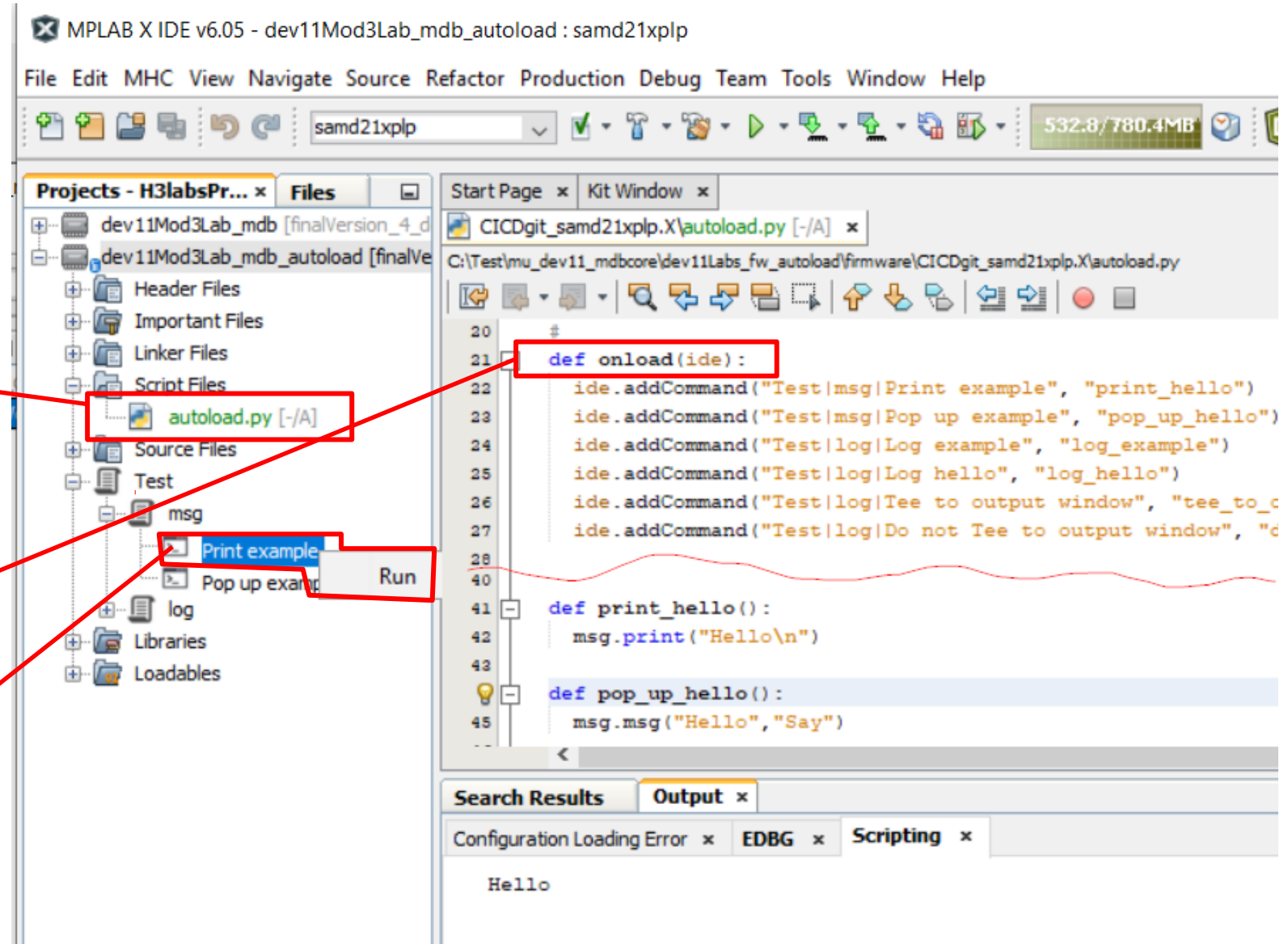
M6: how to enable/use python-scripting

To use python-scripting

- place an `autoload.py` (!name=fix!) file in prj.X\



- MPLABX will automaticall pick up prj.X\autoload.py
- and execute on **project-load**
- Now you can right-click onto the fct to execute it - eg: to run the python-fct 'print_hello()'

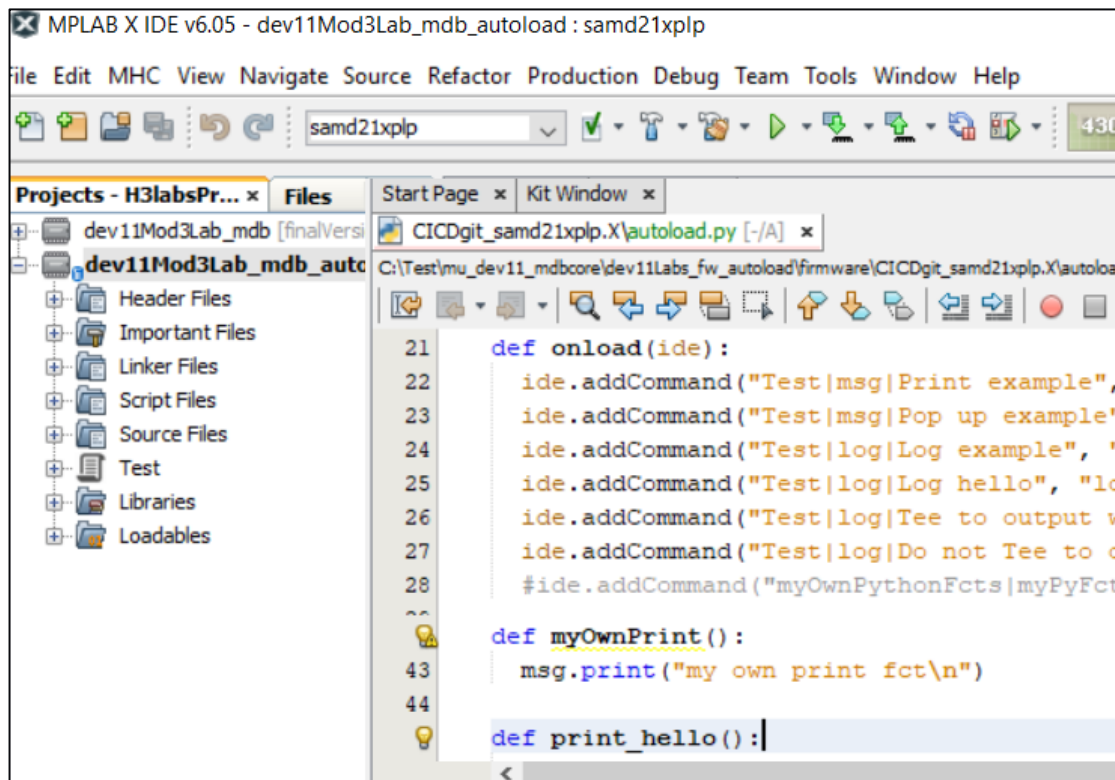


M6: autoload.py potentials

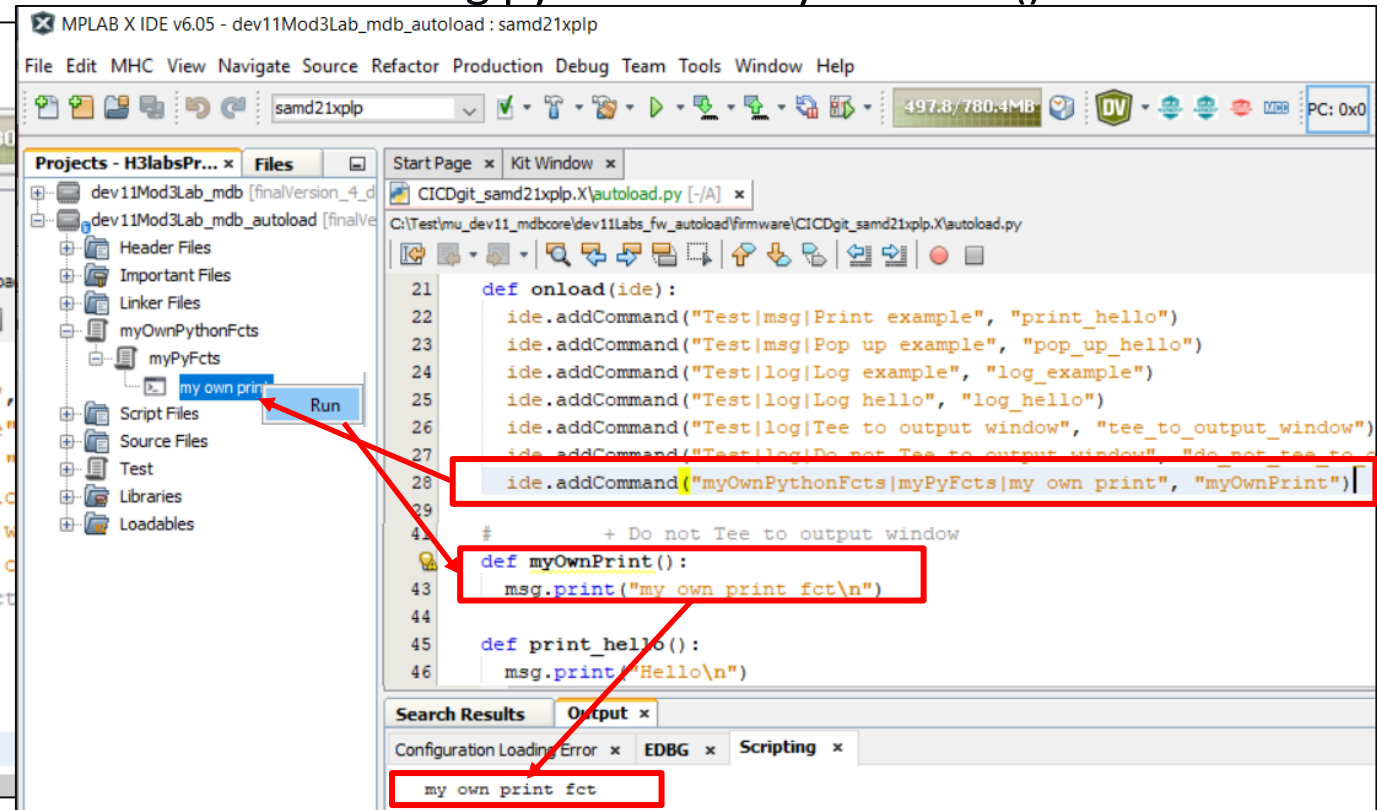
Potential of autoload.py

- Now you've seen that you can place your own python-functions in a file called autoload.py
- Then simply place autoload.py into prj.X\ folder, where MPLABX will automatically pick it up on projectLoad
- With this you can create and execute your own python-functions in very simple way - example below

before



after adding python-fct 'myOwnPrint()'



M6: autoload.py potentials

Potential of autoload.py (continued)

- The comments in autoload.py have some explanations and examples that show what you can do
- You find modules 'msg' or 'log' to print messages into either output-tab (->msg) or the logfile MPLABX uses (->log)
- You will also see modules like 'ide' which are nowhere defined in autoload.py
- but obviously work, so where are they defined?
- These are MDBCore-modules and described in the SDK-docu (->see module1/module4)
- MDBCore-modules/-objects used in autoload.py example
 - msg -> using the MDBCore-class 'msg' to output a message in the Output-Tab
 - log -> using the MDBCore-class 'log' to output messages into the logfile MPLABX uses
 - mem -> the MDBCore-object 'mem' to access any device-memory
 - deb -> the MDBCore-object 'debugger'
- The 'mem' object

To understand its usage you have to know how MPLABX accesses device memory

 - MPLABX uses the computer RAM to cache/buffer access between IDE and real device, eg partial updates
 - "Write32()" writes to PC-cache only whereas "WriteHW32()" writes to PC-cache into real device-memory
- The 'deb' object allows you to use debugger-functions like 'connect()' or 'run()' -> basically the functionality you learned in mod3/mdb and mod4/mdbcs but with autoload.py but from within the IDE

M6: how to extend the MPLABX-python env?

Can you extend the MPLABX-python environment maybe to use any of the many Python-module out?

- you seen how easy it is to create your own python-functions -> simply place them into `autoload.py` and store it in `prj.X\` directory, to use your python-functions
- BUT what if I want to use any of the million python modules out there?
- Is there a pip-installer for the MPLABX-jython-environment?
- No the MPLABX-python-environment is frozen to Jython-v2.7.x capabilities because Jython, Python version with a Java-IF, was developed with Python-v2.7.2 and frozen with no further development and it cannot be extended
- So that's it? Well no, although you cannot extend MPLABX-jython v2.7.x environment, simple workaround is:
 - Install Python-v2.7.x package on your local computer
 - With this you have the pip-installer to add any python-module
 - Any installed python-module will create a folder in `local_pack_folder=<python-installation>\Lib\site-packages\`
 - finally use `'sys.append(<local_pack_folder>')` in `autoload.py` and you can use any locally installed packages
- Next slides show how to use python-module 'intelHex' in `autoload.py` which is helpful if you know that XC-compilers create Intel-hex formatted hexfiles

M6: how to extend the MPLABX-python env?

- As example we add Python-package 'intelhex' which allows access MPLABX-created hex-files
- First install the appropriate local python-version v2.7 – eg get latest v2.7x version from <https://www.python.org/downloads/release/python-2718/> and install into eg: 'C:\Python\v2.7.18\'
- Next use **pip** to see which modules are installed by default -> (DOS)> **pip list**
- Then add the intelhex module with (DOS)> **pip install intelhex** -> doublecheck with (DOS)> **pip list**

```
C:\Users\m16422\AppData\Local\Programs\Python\Python37>c:\Python27\Scripts\pip.exe list
DEPRECATION: Python 2.7 reached the end of its life on January 1st, 2020. Please upgrade your Python as Python 2.7 is no
longer maintained. A future version of pip will drop support for Python 2.7. More details about Python 2 support in pip
, can be found at https://pip.pypa.io/en/latest/development/release-process/#python-2-support
Package      Version
-----
pip          20.0.2
setuptools   28.8.0

C:\Users\m16422\AppData\Local\Programs\Python\Python37>c:\Python27\Scripts\pip.exe install intelhex
DEPRECATION: Python 2.7 reached the end of its life on January 1st, 2020. Please upgrade your Python as Python 2.7 is no
longer maintained. A future version of pip will drop support for Python 2.7. More details about Python 2 support in pip
, can be found at https://pip.pypa.io/en/latest/development/release-process/#python-2-support
Collecting intelhex
  Using cached intelhex-2.2.1-py2.py3-none-any.whl (50 kB)
Installing collected packages: intelhex
Successfully installed intelhex-2.2.1

C:\Users\m16422\AppData\Local\Programs\Python\Python37>c:\Python27\Scripts\pip.exe list
DEPRECATION: Python 2.7 reached the end of its life on January 1st, 2020. Please upgrade your Python as Python 2.7 is no
longer maintained. A future version of pip will drop support for Python 2.7. More details about Python 2 support in pip
, can be found at https://pip.pypa.io/en/latest/development/release-process/#python-2-support
Package      Version
-----
intelhex     2.2.1
pip          20.0.2
setuptools   28.8.0
```

M6: how to extend the MPLABX-python env?

- Now you can locally install Python-v2.7.x or Jython-v2.7.x
- Previous slide showed how to extend MPLABX-autoload.py/-jython functionality through a local Python-v2.7.x installation
- If you want to use Jython be aware of this pitfall



jython

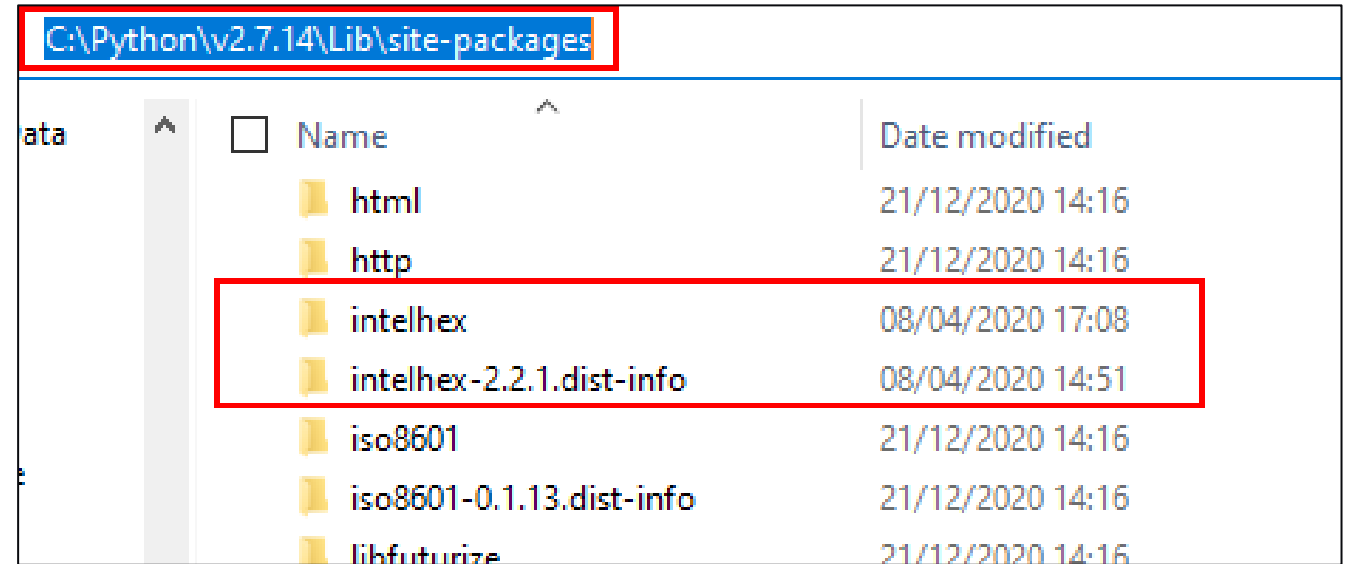
If you install jython into the non-std-path, like you see below 'c:\program files(x86)\...' then you'll get below error, complaining 'cannot load python27.dll..' or alike.

```
C:\Windows\System32\cmd.exe
C:\microchip\0prjsCust\Bosch\ThermoTec\190926_AutocompRegr-demo\SAMC21J\ndbcore2> "C:\Program Files (x86)\Jython\v2.7.1\bin\jython.exe" myTest.py
Error loading Python DLL: C:\Program Files (x86)\Jython\v2.7.1\bin\python27.dll (error code 14001)
C:\microchip\0prjsCust\Bosch\ThermoTec\190926_AutocompRegr-demo\SAMC21J\ndbcore2>
C:\microchip\0prjsCust\Bosch\ThermoTec\190926_AutocompRegr-demo\SAMC21J\ndbcore2> "C:\Program Files\Python\Python27\python.exe" myTest.py
Traceback (most recent call last):
  File "myTest.py", line 6, in <module>
    from con.microchip.ndbcs import Debugger
ImportError: No module named con.microchip.ndbcs
C:\microchip\0prjsCust\Bosch\ThermoTec\190926_AutocompRegr-demo\SAMC21J\ndbcore2>
```

=>Problem finally was that jython-v2.7.x mustbe installed into the default directory: "C:\jython2.7.0\bin\jython.exe"

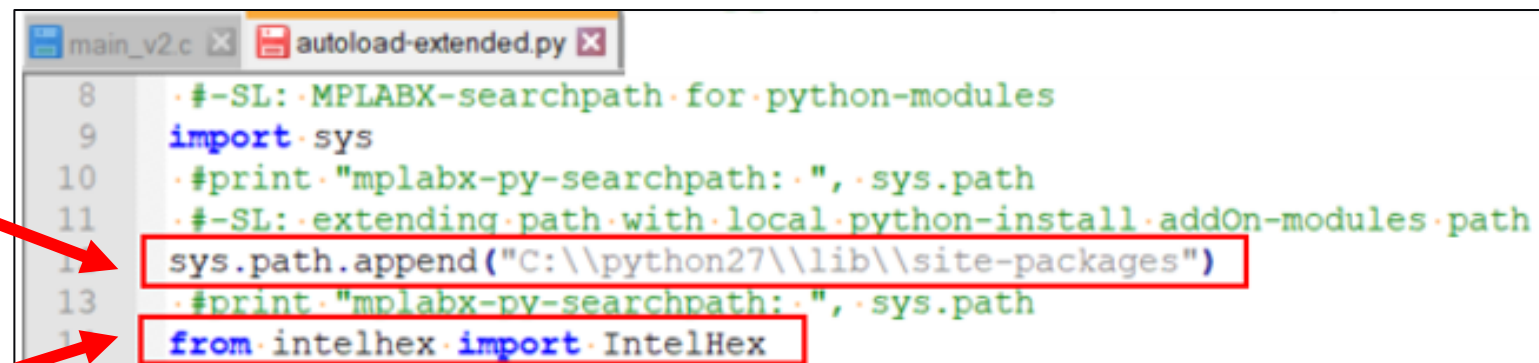
M6: how to extend the MPLABX-python env?

- Once 'intelhex' package is installed with `pip`, you find two new directories in the python-package path 'site-packages\' where addOn packages are installed into



Name	Date modified
html	21/12/2020 14:16
http	21/12/2020 14:16
intelhex	08/04/2020 17:08
intelhex-2.2.1.dist-info	08/04/2020 14:51
iso8601	21/12/2020 14:16
iso8601-0.1.13.dist-info	21/12/2020 14:16
libfuturize	21/12/2020 14:16

- To use this intelhex-module inside `autoload.py`, simply extend the `jython`-searchpath to this local installation with `sys.path.append()` (first need `import sys`)
- Finally use it with (case-sensitive!):
`from intelhex import IntelHex`



```
8  #-SL: MPLABX-searchpath for python-modules
9  import sys
10 #print "mplabx-py-searchpath: ", sys.path
11 #-SL: extending path with local python-install addOn-modules path
12 sys.path.append("C:\\python27\\lib\\site-packages")
13 #print "mplabx-py-searchpath: ", sys.path
14 from intelhex import IntelHex
```

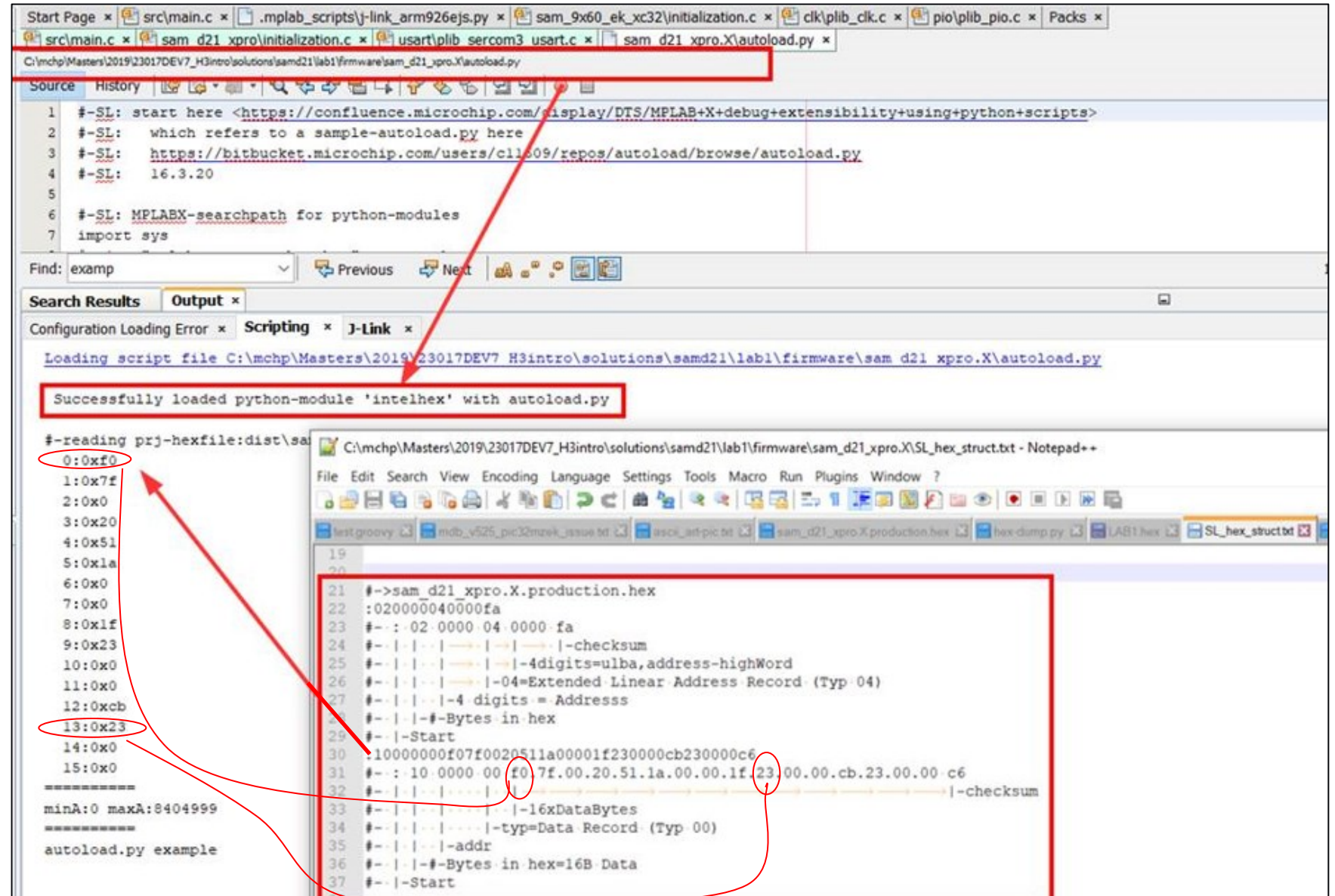
M6: how to extend the MPLABX-python env?

The screenshot displays the MPLABX IDE interface. On the left, the 'Proj...' pane shows a project tree where a new entry 'dump_hex (EDBG-win)' has been added to the 'myCmds' folder. The main editor shows the 'main.c' file with a call to 'ide.addCommand()' for 'dump_hex'. On the right, the 'autoload.py' script is open, showing a new function 'def dump_hex():' that imports 'IntelHex' and reads a hex file. Red arrows indicate the flow from the menu entry to the function definition and then to its execution.

- New entry `dump_hex` in the prj-menu is created with `ide.addCommand()`
- Double-click or right-click+Run to executes the python-function `def dump_hex()`
- `dump_hex()` uses the added python-module `intelhex` to read the hexfile
->see next page

M6: how to extend the MPLABX-python env?

- All `msg.print()` will show up in the output-tab
- Reviewing the original hex file shows that the output is correct but nicely formatted



M6: pythonIF - demo

Brief demo of the addon-python module 'intelhex'

- XC*-compilers create a hex-file which is formatted as Intel-Hex
- The python-module 'intelHex' is not part of the default MPLABX-installation
- You can easily add it though, by installing into your local Python-installation with pip and then use it with 'sys.append(<path-to-local-python-packages>')

=>let's briefly see a demo

M6: pythonIF - Knowledge Check

Summary and knowledge check module6: `autoload.py`

- what is `autoload.py` ?
=> Provides an example how to create+use your own Python-functions to extend MPLABX-functionality
- Can you extend/add any of the million python-modules?
=> Yes you can, by installing the addon-package locally and just reference it with `sys.append(<path-2-local-packages>)`

M6: Overall knowledge check

Summary of whole training on FW-automation with MDBCore (1)

- Part2/compile
 - steps to compile your FW-revisionX to revisionX.elf ?
=> Get FW-revision = git checkout => prjMakefileGenerator.bat => make
- Part3/FW-test automation with `MDBCore`
 - what is `MDBCore` ?
=> A collection of Java-modules that handle all functions needed to download+debug a FW
 - What interfaces can you use to access `MDBCore` ?
=> 3 IFs: `mdb` (`mdb.bat= java -jar mdb.jar`) -> `mdbcs` (`mdbcs.jar`) -> `MDBCore` (Java)
 - Advantages of `mdb.bat` vs `mdbcs` vs `MDBCore` ?
=> `mdb` (simple ,no-install but limits) -> `mdbcs` (solves `mdb`-limits, but requires install+config, wraps major `MDBCore`-functions, to ease `MDBCore`-usage, only scripting-knowledge) -> `MDBCore` (need good Java-skills)
- part6/python:
 - what is `autoload.py` ?
=> Allows to use your own Python-fuctions to extend MPLABX-functionality – extend with any Python-package

End Mod6: autoload.py

End of Mod6:

**extending MPLABX-functionality with *Python*
(*autoload.py*)**

END of this training on:

Automated FW-testing using *MDBCore*